

Adattudomány MSc – záróvizsga tételek

Bence Dáné

2026

Tartalomjegyzék

1. Az adattudomány elméleti háttéréhez kapcsolódó alapismeretek.	1
1.1. 1. tétel: információbiztonsági alapok	1
1.2. 2. tétel: hitelesítés, nyomonkövethetőség és biztonságos üzemeltetés .	5
1.3. 3. tétel: gépi tanulási alapismeretek	14
2. Az adattudomány gyakorlati háttéréhez kapcsolódó alapismeretek.	18
2.1. 4. tétel: modellkiértékelés, SVM, klaszterezés, dimenziócsökkentés, ajánlórendszerek és MapReduce	18

1. Az adattudomány elméleti háttéréhez kapcsolódó alapismeretek.

1.1. 1. tétel: információbiztonsági alapok

Az információbiztonság alapfogalmai (CIA-hármas); a kiberbiztonság eszközei és céljai; kártékony programok, támadási technológiák; hozzáférés-szabályozás (DAC, MAC, RBAC, ABAC, CBAC); hozzáférés-szabályozás elosztott rendszerekben.

Információbiztonsági alapfogalmak

Az információbiztonság célja az információ és az információs rendszerek védelme a jogosulatlan hozzáféréssel, módosítással, megsemmisítéssel, szolgáltatás-kieséssel és visszaéléssel szemben. A klasszikus alapmodell a **CIA-hármas**:

- **Confidentiality – bizalmasság:** az információ csak jogosult személy, folyamat vagy rendszer számára legyen hozzáférhető. Példa: titkosítás, hozzáférés-vezérlés, jogosultságkezelés.
- **Integrity – sértetlenség:** az adat pontos, teljes és jogosulatlanul nem módosított. Példa: hash, MAC, digitális aláírás, tranzakciós naplózás.

- **Availability – rendelkezésre állás:** a jogosult felhasználók a szükséges időben elérik a rendszert vagy szolgáltatást. Példa: redundancia, mentés, terheléselosztás, DoS-védelem.

Ezeket gyakran kiegészítik további biztonsági célokkal: **hitelesség, elszámoltathatóság, nyomonkövethetőség és letagadhatatlanság**. A hitelesség azt jelenti, hogy a forrás vagy entitás valóban az, akinek állítja magát; az elszámoltathatóság és nyomonkövethetőség pedig azt, hogy utólag rekonstruálható, ki mit tett.

A kiberbiztonság céljai és eszközei

A kiberbiztonság az információs rendszerek, hálózatok, alkalmazások, hardverek, adatok és szolgáltatások védelmét jelenti. A védelem nem csak technológia: emberek, folyamatok és szervezeti szabályok együttese.

Fő célja a szervezet működésének, adatainak és szolgáltatásainak védelme úgy, hogy a kockázatok elfogadható szintre csökkenjenek. Ez nem azt jelenti, hogy minden támadás teljesen kizárható, hanem azt, hogy a támadási felületet csökkentjük, a hibákat időben felismerjük, és incidens esetén gyorsan helyreállítunk. Ennek eszközei a megelőző kontrollok (például erős hitelesítés, titkosítás, jogosultságkezelés), a detektív kontrollok (naplózás, monitorozás, IDS/IPS), valamint a korrektív kontrollok (mentés, incidenskezelés, helyreállítás). A gyakorlatban rétegzett védelemre van szükség, mert egyetlen eszköz hibája esetén a többi kontrollnak kell csökkentenie a kárt.

Terület	Tipikus eszközök és célok
Rendszerbiztonság	Operációs rendszer jogosultságai, folyamat- és memóriavédelem, virtualizáció, izoláció, Trusted Execution Environment.
Kriptográfia	Titkosítás, hash, MAC, digitális aláírás, kulcskezelés, tanúsítványok.
AAA	Hitelesítés, feljogosítás, naplózás/elszámoltathatóság.
Hálózatbiztonság	Tűzfal, IDS/IPS, VPN, DMZ, szegmentáció, biztonságos protokollok.
Hardverbiztonság	Root of trust, TPM/HSM, biztonságos kulcstárolás, valódi véletlenszám-generátor, PUF.
Szoftverbiztonság	Secure SDLC, threat modeling, secure coding, review, tesztelés, dependency management.
Emberi és szervezeti védelem	Usable security, képzés, szabályzatok, kockázatmenedzsment, GDPR, NIS2.

Kártékony programok és támadási technológiák

Kártékony program minden olyan szoftver, amelyet a támadó jogosulatlan hozzáférés, adatlopás, károkozás, rejtőzködés vagy zsarolás céljából használ.

A kártékony programok gyakran nem önmagukban, hanem egy támadási lánc részeként jelennek meg: bejutás után perzisztenciát építenek ki, jogosultságot növelnek, oldalirányban mozognak a hálózatban, majd adatot lopnak, rombolnak vagy zsarolnak. A modern támadások sokszor technikai és emberi elemeket kombinálnak, például phishinggel szerzik meg a belépési pontot, majd sérülékenység-kihasználással terjeszkednek tovább. Védekezésben fontos a naprakész patch-elés, a végpontvédelem, a jogosultságok minimalizálása, a hálózati szegmentáció, a biztonsági mentés és a felhasználók képzése. A ransomware elleni védelemben különösen lényeges az offline vagy elkülönített mentés, mert a támadó gyakran a mentéseket is megpróbálja törölni vagy titkosítani.

- **Vírus:** gazdaprogramhoz kapcsolódik, annak futásával terjed.
- **Féreg:** önállóan, hálózaton keresztül terjedhet.
- **Trójai:** hasznos programnak álcázza magát, de rejtett káros funkciót tartalmaz.
- **Ransomware:** titkosítja vagy zárolja az adatokat, majd váltságdíjat követel.
- **Spyware/keylogger:** információt, jelszót, billentyűleütést gyűjt.
- **Rootkit/backdoor:** tartós rejtett hozzáférést biztosít a támadónak.
- **Botnet malware:** kompromittált gépeket központi vezérlés alá von, például DDoS-hoz vagy spamhez.

Tipikus támadási technológiák:

- **Social engineering és phishing:** a felhasználó megtévesztése hitelesítő adatok vagy művelet kicsalására.
- **Brute force és credential stuffing:** jelszavak próbálgatása vagy kiszivárgott jelszavak újrafelhasználása.
- **DoS/DDoS:** a rendelkezésre állás támadása túlterheléssel.
- **Man-in-the-middle:** kommunikáció lehallgatása vagy módosítása két fél között.
- **Injection:** például SQL, command vagy script injection, ahol a támadó bemenetként kódot juttat a rendszerbe.
- **Privilege escalation:** alacsonyabb jogosultságból magasabb jogosultság megszerzése.
- **Exploit és sérülékenység-kihasználás:** szoftverhiba célzott kihasználása.

Hozzáférés-szabályozás alapjai

A hozzáférés-szabályozás célja annak eldöntése, hogy egy **alany** (felhasználó, folyamat, szolgáltatás) végrehajthat-e egy **műveletet** egy **objektumon** (fájl, adatbázis, API, szolgáltatás). A döntés alapja egy szabályrendszer, azaz **policy**.

Az AAA-modell:

- **Authentication – hitelesítés:** ki kezdeményezte a kérést?
- **Authorization – feljogosítás:** jogosult-e a kért műveletre?
- **Accounting – naplózás:** mi történt, mikor, milyen eredménnyel?

Hozzáférés-szabályozási komponensek:

- **PEP – Policy Enforcement Point:** kikényszeríti a döntést, például API gateway vagy operációs rendszer.

- **PDP – Policy Decision Point:** meghozza az engedélyezési döntést.
- **PIP – Policy Information Point:** attribútumokat szolgáltat, például szerepkör, eszközállapot, idő, hely.
- **Audit log:** a döntés és művelet nyoma.

Hozzáférés-szabályozási modellek

A hozzáférés-szabályozási modellek azt határozzák meg, milyen elv alapján születik meg az engedélyezési döntés. A választott modell mindig kompromisszum a rugalmasság, az adminisztrálhatóság és a biztonsági szigor között: egy kis csoportban elég lehet a DAC, nagyvállalati környezetben tipikus az RBAC, míg érzékeny vagy dinamikus rendszerekben gyakran ABAC vagy MAC szükséges. A legfontosabb általános elv a legkisebb jogosultság elve, vagyis az alany csak azt a hozzáférést kapja meg, amely a feladatához valóban szükséges. A modellek a gyakorlatban keveredhetnek is, például egy vállalati rendszer szerepköröket használ, de bizonyos műveleteket időhöz, eszközállapothoz vagy kockázati szinthez köt.

Modell	Lényege	Tipikus használat / megjegyzés
DAC	A tulajdonos dönt arról, ki férhet hozzá az objektumhoz.	Rugalmas, de könnyen túlengedélyezhető; klasszikus fájlrendszer-jogosultságok.
MAC	Központilag meghatározott címkék és biztonsági szintek alapján dönt.	Katonai/államigazgatási környezet; a tulajdonos nem írhatja felül.
RBAC	Jogosultságok szerepkörök-höz tartoznak, felhasználók szerepköröket kapnak.	Vállalati rendszerek; munkakör alapú jogosultságkezelés.
ABAC	Felhasználó-, erőforrás- és környezeti attribútumok alapján dönt.	Finomabb szabályok: idő, hely, eszközállapot, kockázati szint.
CBAC	A végrehajtott kód eredete, identitása, hash-e vagy aláírása alapján dönt.	Mobilkód, sandbox, futtatható állományok kontrollja.

Fontos kiegészítés: a szabályozás megvalósítható hozzáférési mátrixszal, ACL-lel vagy képességlistával. Az **ACL** objektumként tárolja, ki mit tehet; a **capability** alanyonként tárolja, milyen objektumhoz milyen joga van.

Hozzáférés-szabályozás elosztott rendszerekben

Elosztott rendszerben a felhasználók, szolgáltatások és erőforrások több gépen, hálózati zónában vagy szervezetben lehetnek. Ezért a hozzáférési döntéshez szükséges identitást, attribútumokat, szabályokat és bizonyítékokat biztonságosan kell továbbítani.

Fő kihívások:

- **Bizalmi határok:** más szervezet vagy szolgáltatás állításait kell elfogadni.
- **Központi vs. lokális döntés:** központi policy könnyebben kezelhető, de kiesési pont lehet; lokális döntés skálázhatóbb, de inkonzisztens lehet.
- **Attribútumok frissessége:** szerepkör, jogosultság vagy token visszavonása gyorsan érvényesüljön.
- **Kommunikáció védelme:** titkosított, hitelesített csatorna szükséges.
- **Naplózás és audit:** több rendszer eseményeit kell időben és tartalmilag korrelálni.

Tipikus megoldások:

- **Kerberos:** jegyalapú, szimmetrikus kulcsos hitelesítés egy bizalmi tartományon belül.
- **SAML, OAuth2, OpenID Connect:** token/assertion alapú föderált identitás és hozzáférés webes és API-környezetben.
- **mTLS és tanúsítványok:** szolgáltatás-szolgáltatás kommunikáció kölcsönös hitelesítése.
- **Attribútum-alapú titkosítás:** az adat úgy titkosítható, hogy csak megfelelő attribútumokkal rendelkező kulcs birtokosa férjen hozzá.
- **Zero Trust szemlélet:** nincs automatikus bizalom belső hálózati pozíció alapján; minden hozzáférést hitelesíteni, autorizálni és naplózni kell.

1.2. 2. tétel: hitelesítés, nyomonkövethetőség és biztonságos üzemeltetés

Hitelesítés, felhasználó-hitelesítés; hitelesítés elosztott rendszerekben; nyomonkövethetőség; biztonságos üzemeltetés és incidenskezelés; Monitor–Analyze–Plan–Execute–Knowledge (MAPE-K); rendeletek, szabványok.

A tétel helye az információbiztonságban

Az információbiztonság célja az információ és az információs rendszerek bizalmosságának, sértetlenségének, rendelkezésre állásának, hitelességének, elszámoltathatóságának és nyomonkövethetőségének biztosítása. A jelen tétel ezek közül főként a hitelességre, a hozzáférésmenedzsmentre, az események utólagos bizonyíthatóságára, valamint a biztonságos üzemeltetésre koncentrál.

A hozzáférésmenedzsment klasszikus felbontása az **AAA-modell**:

- **Authentication – hitelesítés:** annak ellenőrzése, hogy az entitás valóban az, akinek állítja magát.
- **Authorization – feljogosítás:** annak eldöntése, hogy a hitelesített entitás milyen erőforráshoz, milyen művelettel férhet hozzá.
- **Accounting – naplózás, elszámoltathatóság:** annak rögzítése, hogy ki, mikor, honnan, milyen műveletet hajtott végre, illetve ez milyen eredménnyel járt.

Vizsgán fontos különbséget tenni az *azonosítás*, a *hitelesítés* és a *feljegyzítés* között. Az azonosítás pusztán az identitás megadása, például felhasználónévvel. A hitelesítés ennek alátámasztása, például jelszóval, tokenel vagy biometrikus adattal. A feljegyzítés már az üzleti vagy biztonsági szabályok alapján dönt a hozzáférésről.

Hitelesítés és felhasználó-hitelesítés

Hitelesség és entitáshitelesítés A hitelesség azt fejezi ki, hogy valaminek a forrása az, amit megjelöltek, és tartalma nem módosult illetéktelenül. Két gyakori értelmezése:

- **Entitáshitelesítés:** egy felhasználó, szolgáltatás, eszköz vagy folyamat identitásának ellenőrzése.
- **Üzenethitelesítés:** annak ellenőrzése, hogy az üzenet adott forrásból származik és nem változott meg. Erre használható például MAC vagy digitális aláírás.

Felhasználó-hitelesítéskor a rendszer nem magát a személyt, hanem egy *hitelesítő adatot* ellenőriz. Ezért a hitelesítés biztonsága mindig függ attól, mennyire nehéz az adott hitelesítő adatot megszerezni, hamisítani, újrajátszani vagy kikényszeríteni.

Hitelesítési faktorok A hitelesítési módszerek alapvetően három fő faktorra épülnek:

- **Tudás alapú faktor:** amit a felhasználó tud, például jelszó, PIN, jelmondat.
- **Birtoklás alapú faktor:** amivel a felhasználó rendelkezik, például okoskártya, hardverkulcs, mobilalkalmazás, TOTP-token.
- **Biometrikus vagy inherens faktor:** ami a felhasználóra jellemző, például ujjlenyomat, írisz, arc, hang, gépelési ritmus.

A többfaktoros hitelesítés akkor erős, ha a faktorok egymástól független kompromittálási csatornákkal rendelkeznek. Például a jelszó és az SMS-kód két faktor ugyan, de SIM-csere, adathalászat vagy rosszindulatú mobilalkalmazás esetén egyszerre is sérülhetnek. Erősebb megoldás a hardveres biztonsági kulcs vagy a platform-hitelesítő, különösen ha a hitelesítés kriptográfiailag kötődik az adott szolgáltatás domainjéhez.

Jelszavas hitelesítés A jelszó előnye, hogy olcsó, egyszerűen bevezethető és nem igényel külön eszközt. Hátránya, hogy a felhasználók gyakran újrahasznosítják, gyenge jelszót választanak, vagy adathalász támadásban kiadják. A biztonságos jelszókezelés főbb elvei:

- A jelszavakat nem szabad nyílt szövegben tárolni; szózott, lassú, memóriaigényes jelszó-kivonatoló függvény használata szükséges.
- A jelszóhossz fontosabb, mint a mesterségesen bonyolult, de rövid összetételi szabályok.
- Támogatni kell a jelszókezelők használatát és tiltani kell az ismertén kiszivárgott jelszavakat.
- Kritikus rendszereknél jelszó önmagában nem elegendő, többfaktoros hitelesítés szükséges.

Tokenek, okoskártyák és tanúsítványok A birtoklás alapú hitelesítés egyik tipikus formája az okoskártya vagy hardveres token. Ezek gyakran kriptográfiai kulcsot tárolnak, amelyet a felhasználó nem tud közvetlenül kiolvasni, csak műveletet kérhet vele. A hitelesítés történhet kihívás-válasz protokollal: a szerver egy friss, egyszer használatos kihívást küld, a token pedig a privát kulccsal bizonyítja birtoklását. Így a privát kulcs nem kerül át a hálózaton.

Digitális tanúsítvány esetén egy hitelesítésszolgáltató köti össze a nyilvános kulcsot egy identitással. Ez gép-gép kommunikációban, VPN-ben, TLS-ben, közigazgatási vagy vállalati rendszerekben különösen fontos.

Biometrikus hitelesítés A biometria kényelmes, mert a felhasználónak nem kell jelszót megjegyeznie vagy külön eszközt hordania. Statikus biometria például az ujjlenyomat, írisz, retina vagy arc; dinamikus biometria például az aláírás, hang, gépelési ritmus vagy járásminta.

Biztonsági szempontból a biometria speciális: jelszóval ellentétben nem cserélhető könnyen, és adatvédelmi kockázatot hordoz. Emiatt a biometrikus sablont védeni kell, célszerű helyben, biztonságos hardveres tárolóban feldolgozni, és nem központi adatbázisban nyíltan tárolni. A biometrikus rendszerek hibái sem binárisak: beszélni kell hamis elfogadási arányról és hamis elutasítási arányról.

A hitelesítés tipikus támadásai A felhasználó-hitelesítést több támadási osztály fenyegeti:

- **Adathalászat:** a támadó hamis felületre irányítja a felhasználót, és megszerzi a hitelesítő adatot.
- **Credential stuffing:** korábban kiszivárgott jelszó-felhasználónév párok automatizált kipróbálása más szolgáltatásokon.
- **Brute force és szótáras támadás:** gyenge vagy kiszámítható jelszavak próbálgatása.
- **Replay támadás:** korábbi hitelesítési üzenet újrajátszása, ha nincs friss nonce, időbélyeg vagy munkamenet-kötés.
- **Session hijacking:** a hitelesítés után létrejött session token megszerzése.
- **MFA fatigue:** a felhasználó ismételt push-kérések hatására jóváhagy egy támadói belépést.

A védekezés réteges: erős jelszótárolás, MFA, adathalászatnak ellenálló hitelesítők, kockázatalapú belépésvizsgálat, eszköz- és helyfüggő anomáliafigyelés, rate limiting, account lockout, biztonságos session-kezelés és részletes naplózás.

Hitelesítés elosztott rendszerekben

Miért nehezebb elosztott környezetben? Elosztott rendszerben a szolgáltatások, felhasználók, identitásszolgáltatók és erőforrások több gépen, hálózati zónában, szervezetben vagy felhőszolgáltatásban helyezkednek el. Ilyenkor a hitelesítés problémái kibővülnek:

- a felek gyakran nem ugyanabban a bizalmi tartományban vannak;

- a hálózat nem tekinthető megbízhatónak;
- szükség van kölcsönös hitelesítésre, nemcsak a felhasználó, hanem a szerver oldalán is;
- a hitelesítési állapotot tovább kell vinni több szolgáltatás között;
- kezelni kell a tokenek élettartamát, visszavonását és továbbadását.

Az elosztott rendszerekben ezért gyakori a központi vagy föderált identitáskezelés: a felhasználó egy identitásszolgáltatónál hitelesít, majd a szolgáltatások megbíznak az identitásszolgáltató által kiadott jegyben, állításban vagy tokenben.

Kerberos A Kerberos klasszikus, szimmetrikus kulcsokra épülő hálózati hitelesítési protokoll. Célja, hogy a kliens és a szolgáltatás kölcsönösen hitelesíteni tudja egymást úgy, hogy a felhasználó jelszava vagy annak lenyomata ne menjen át a hálózaton.

Fő szereplők:

- **Kliens:** a felhasználó munkaállomása vagy folyamata.
- **Application Server:** az igénybe venni kívánt szolgáltatás.
- **Authentication Server (AS):** a felhasználó kezdeti hitelesítését végzi.
- **Ticket Granting Server (TGS):** szolgáltatói jegyeket ad ki.
- **Key Distribution Center (KDC):** az AS és TGS együttese.

A működés röviden:

1. A kliens az AS-től **Ticket Granting Ticketet** (TGT) kér.
2. Az AS a felhasználó jelszavából származtatott kulccsal titkosítva küldi vissza a TGT használatához szükséges session keyt.
3. Ha a felhasználó helyes jelszót adott meg, a kliens vissza tudja fejteni ezt, de a jelszó nem utazott a hálózaton.
4. A kliens a TGT-vel a TGS-től szolgáltatói jegyet kér.
5. A szolgáltatói jeggyel és egy rövid érvényességű authenticatorral fordul a konkrét szerverhez.
6. Kölcsönös hitelesítés esetén a szerver is bizonyítja, hogy ismeri a megfelelő session keyt.

A Kerberos erőssége a single sign-on jelleg és a jelszó hálózaton kívül tartása. Kockázata, hogy a KDC kiemelten kritikus komponens, a jegyek időhöz kötöttek, ezért pontos időszinkron kell, és a TGT megszerzése nagy értékű támadói cél.

OAuth 2.0 és OpenID Connect Az OAuth 2.0 elsősorban **felhatalmazási** keretrendszer: azt teszi lehetővé, hogy egy kliensalkalmazás a felhasználó erőforrásaihoz korlátozott hozzáférést kapjon anélkül, hogy a felhasználó jelszavát megkapná. Tipikus szereplők:

- **Resource Owner:** az erőforrás tulajdonosa, rendszerint a felhasználó.
- **Client:** az alkalmazás, amely hozzáférést kér.

- **Authorization Server:** hitelesít, hozzájárulást kezel, token-t ad ki.
- **Resource Server:** az API vagy szolgáltatás, amely a token-t ellenőrzi.

Az OAuth hozzáférési token-t ad ki, amely meghatározott hatókörrel és élettartammal használható. Az **OpenID Connect** erre épül, és hitelesítési réteget ad hozzá: az Authorization Server digitálisan aláírt ID token-t ad ki, amely állításokat tartalmaz a felhasználó identitásáról.

Fontos vizsgapont: az OAuth önmagában nem felhasználó-hitelesítési protokoll, hanem felhatalmazási keretrendszer. Ha bejelentkeztetésre használjuk, OpenID Connectre vagy más identitásrétegre van szükség.

Föderált identitás és SSO Föderált identitáskezelésnél egy szervezet vagy szolgáltatás megbízza egy külső identitásszolgáltatóban. A felhasználó az IdP-nél hitelesít, a szolgáltatás pedig aláírt állítás vagy token alapján dönt. Előnyei:

- központi felhasználó-életciklus-kezelés;
- kevesebb jelszó és kevesebb helyi fiók;
- egységes MFA és hozzáférési szabályok;
- könnyebb auditálhatóság.

Kockázatai:

- az identitásszolgáltató kiesése vagy kompromittálása nagy hatású;
- rosszul beállított redirect URI, token-validáció vagy aláírásellenőrzés súlyos sebezhetőség;
- túl hosszú token-élettartam vagy gyenge visszavonási mechanizmus támadói mozgásteret ad.

Hozzáférésvezérlés kapcsolódása A hitelesítés után a rendszernek el kell döntenie, hogy a hitelesített entitás mire jogosult. Gyakori modellek:

- **DAC:** a tulajdonos vagy felhasználó által vezérelt hozzáférés.
- **MAC:** központi, címkéken és biztonsági szinteken alapuló hozzáférés.
- **RBAC:** szerepkörök alapján adott jogosultságok.
- **ABAC:** attribútumok alapján hozott döntés, például felhasználói tulajdonság, eszközállapot, napszak, kockázati szint.

Modern rendszerekben gyakori a Zero Trust szemlélet: nincs automatikus bizalom belső hálózati pozíció alapján, minden hozzáférést folyamatosan hitelesíteni, autorizálni és naplózni kell.

Nyomonkövethetőség, naplózás és audit

A nyomonkövethetőség célja A nyomonkövethetőség olyan folyamatok és technikai megoldások összessége, amelyek az események bekövetkezése után lehetővé teszik a történetek rekonstrukcióját. Ide tartozik a rendszeres audit, a technikai logelemzés, a megfelelőségi ellenőrzés és az incidens utáni vizsgálat.

Egy jó naplózási rendszer meg tudja válaszolni:

- ki vagy mi kezdeményezte az eseményt;

- mikor és honnan történt;
- milyen erőforrást érintett;
- milyen művelet történt;
- sikeres vagy sikertelen volt-e;
- milyen kapcsolódó események előzték meg vagy követték.

Naplóforrások Biztonsági üzemeltetésben fontos naplóforrások:

- operációs rendszer naplók;
- alkalmazásnaplók;
- hitelesítési és jogosultsági naplók;
- hálózati eszközök syslog eseményei;
- tűzfal, VPN, WAF, IDS/IPS riasztások;
- felhőszolgáltatók auditlogjai;
- adatbázis-hozzáférési naplók;
- végpontvédelmi és EDR-események.

Az audit policy határozza meg, hogy mely eseményeket kell rögzíteni. Tipikus példa a sikeres és sikertelen bejelentkezés, jogosultságváltoztatás, adminisztrátori művelet, konfigurációmódosítás vagy érzékeny adathozzáférés.

Bizonyítéki érték és naplóvédelem A naplózás értéke annyi, amennyire a napló megbízható. Incidens esetén a támadó gyakran megpróbálja törölni vagy módosítani a terhelő bejegyzéseket. Ezért a naplók védelmének fő követelményei:

- **Integritás:** a napló utólagos módosítása legyen észlelhető.
- **Időhitelesség:** legyen pontos időszinkron, például NTP-vel, védett időforrással.
- **Elkülönített tárolás:** a napló ne csak azon a rendszeren legyen, amelyet a támadó kompromittált.
- **Hozzáférésvédelem:** a naplókat csak korlátozott szerepkörök érhessek el.
- **Megőrzés és törlési rend:** a retention policy feleljen meg üzleti, jogi és adatvédelmi követelményeknek.

Kriptográfiai védelemre használható hash-lánc, digitális aláírás, WORM-tároló vagy elosztott naplózás. A hash-lánc lényege, hogy minden bejegyzés tartalmazza az előző bejegyzés lenyomatát, így egy köztes módosítás a lánc későbbi részét is érvénytelenné teszi.

Naplóelemzés problémái A naplók mennyisége nagy, a bejegyzések jelentős része biztonsági szempontból irreleváns. Emiatt automatizált feldolgozás, normalizálás, korreláció és priorizálás kell. A gépi tanulás segíthet anomáliák felismerésében, de nem helyettesíti az üzleti kontextust és a szakértői értelmezést. A vizualizáció abban segít, hogy az operátor figyelme a legfontosabb eseményekre irányuljon.

Biztonságos üzemeltetés

SOC, IDS/IPS, SIEM és SOAR A biztonságos üzemeltetés célja, hogy a szervezet folyamatosan képes legyen fenyegetéseket észlelni, elemezni, kezelni és a működést helyreállítani. Ennek intézményi formája gyakran a **Security Operations Center** (SOC), amely technológiát, folyamatot és emberi elemzői munkát kapcsol össze.

Fontos komponensek:

- **IDS:** behatolásészlelő rendszer; eseményekből vagy hálózati forgalomból riasztást generál.
- **IPS/IDPS:** behatolásmegelőző rendszer; inline módon forgalmat is blokkolhat.
- **SIEM:** biztonsági információ- és eseménymenedzsment; naplókat gyűjt, normalizál, korrelál és riaszt.
- **SOAR:** security orchestration, automation and response; automatizált válaszleépéseket, playbookokat és integrációkat kezel.
- **CTI:** cyber threat intelligence; külső és belső fenyegetési információkat ad, például IOC-eket, TTP-eket, sérülékenységi adatokat.

A SIEM egyik fontos feladata az alert-korreláció: csökkenti az elemzőre zúduló riasztások számát, kontextust ad, összeköti az összefüggő eseményeket, és kiszűri a hamis pozitív riasztásokat.

Architektúrális szempontok Üzemeltetési szempontból a védelmi architektúrának nem szabad kizárólag peremvédelmi eszközökre, például tűzfalra épülnie. A DMZ, hálózati szegmentáció, végpontvédelem, naplógyűjtés, sérülékenységmenedzsment és hozzáférésvezérlés együtt ad értelmezhető védelmet.

Tipikus architektúrális elvek:

- kritikus rendszerek elkülönítése;
- menedzsment interfészek külön védelme;
- minimális jogosultság elve;
- változáskezelés és konfigurációmenedzsment;
- mentések rendszeres tesztelése;
- sérülékenységek priorizált javítása;
- incidensre kész kommunikációs és döntési rend.

Incidenskezelés

Az incidens fogalma Biztonsági incidensnek tekinthető minden olyan esemény vagy eseménysorozat, amely sérti vagy veszélyezteti a bizalmasságot, sértetlenséget, rendelkezésre állást, hitelességet vagy az üzleti működés folytonosságát. Nem minden riasztás incidens, és nem minden incidens jelent azonos súlyosságú kockázatot.

Incidenskezelési életciklus Az órai anyag három nagy tevékenységet emel ki: felkészülés, kezelés, lezárás utáni tanulás. Ezek részletesen:

1. **Felkészülés:** szabályzatok, szerepkörök, CSIRT vagy incidenskezelő csapat,

kommunikációs csatornák, kontaktlista, eszközök, naplózás, mentések, gyakorlatok.

2. **Észlelés és elemzés:** riasztások, naplók, felhasználói bejelentések és CTI alapján annak eldöntése, történt-e incidens, mi az érintett kör, mi a súlyosság.
3. **Korlátozás:** a további kár megakadályozása, például hálózati izolációval, fiókletiltással, tűzfalszabállyal vagy szolgáltatás ideiglenes leállításával.
4. **Felszámolás:** rosszindulatú kód, támadói hozzáférés, sérülékeny konfiguráció vagy kompromittált hitelesítő adat eltávolítása.
5. **Helyreállítás:** rendszerek visszaállítása, fokozott monitorozás és óvatos visszakapcsolás.
6. **Utókövetés:** tanulságok, jelentés, kontrollok javítása, képzés, playbookok frissítése.

Az incidenskezelés nem tisztán technikai feladat. Jogi, kommunikációs, vezetői, adatvédelmi és üzletmenet-folytonossági döntéseket is igényel. Különösen fontos, hogy a szervezet előre tudja, mikor kell hatóságot, ügyfelet, partnert vagy felügyeleti szervet értesíteni.

Bizonyítékkezelés Incidensnél a bizonyítékok minősége döntő. A bizonyítékkezelés során figyelni kell:

- a bizonyíték forrására és integritására;
- a hozzáférési láncra, vagyis ki, mikor, mit kezelt;
- a változtatásmentes mentésre;
- a naplók, memóriaképek, lemezképek és hálózati forgalmak időbeli összekapcsolására;
- arra, hogy a helyreállítás ne törölje el a vizsgálathoz szükséges adatokat.

MAPE-K modell

A modell jelentése A MAPE-K az autonóm és biztonsági üzemeltetésben használt visszacsatolási modell:

- **Monitor:** adatgyűjtés eseményekből, naplók, hálózati forgalomból, végpontokból, alkalmazásokból.
- **Analyze:** a begyűjtött nyomok értelmezése, anomália- és visszaélésdetektálás, riasztásminősítés.
- **Plan:** válaszstratégia kiválasztása, priorizálás, döntéstámogatás.
- **Execute:** ellenintézkedések végrehajtása, például blokkolás, izoláció, fiókletiltás, szabálymódosítás.
- **Knowledge:** tudásbázis, sérülékenységi adatbázis, playbookok, fenyegetési intelligencia, tanulságok.

MAPE-K a biztonsági üzemeltetésben A MAPE-K jól megfeleltethető a SOC működésének. A Monitor réteghez tartoznak a szenzorok, naplók és adatfolyamok. Az Analyze réteghez tartozik a detektálási logika, például signature-alapú vagy anomália-alapú elemzés. A Plan rétegben a SIEM és döntéstámogató folyamatok segítenek meghatározni, hogy a riasztás valódi incidens-e és milyen válasz kell. Az Execute rétegben az IDPS, tűzfal, EDR vagy SOAR végrehajtja a kiválasztott lépéseket. A Knowledge réteghez tartozik a CTI, CVE, CVSS, CWE, CERT/CSIRT ajánlások, ISAC információmegosztás és a szervezeti tanulságok.

MAPE-K elem	Biztonsági példa	Kockázat
Monitor	Syslog, EDR, NetFlow, alkalmazásnapló, felhő audit-log	Hiányos vagy manipulált naplózás
Analyze	Korreláció, anomáliadetektálás, IOC-illesztés	Hamis pozitív és hamis negatív riasztások
Plan	Incidensprioritás, playbook kiválasztása	Lassú döntés vagy rossz súlyossági besorolás
Execute	IP-blokkolás, host izoláció, fiókletiltás	Üzleti működés indokolatlan megszakítása
Knowledge	CTI, CVE/CVSS/CWE, tanulságok	Elavult tudásbázis, nem frissített playbook

1. táblázat. MAPE-K elemek biztonsági üzemeltetési értelmezése

Rendeletek és szabványok

Európai és magyar szabályozási környezet GDPR. Az általános adatvédelmi rendelet a személyes adatok kezelésére vonatkozik. Incidenskezelésben különösen fontos a személyes adat megsértésének bejelentése: ha az incidens valószínűsíthetően kockázattal jár az érintettek jogaira és szabadságaira, a felügyeleti hatóságot indokolatlan késedelem nélkül, lehetőség szerint 72 órán belül értesíteni kell. Magas kockázat esetén az érintettek tájékoztatása is szükséges lehet.

NIS2. A NIS2 irányelv célja az EU egészében magas közös kiberbiztonsági szint kialakítása. A tagállamoknak 2024. október 17-ig kellett átültetniük, a szabályok 2024. október 18-tól alkalmazandók. Lényeges és fontos szervezetekre ír elő kockázatkezelési, incidensbejelentési, vezetői felelősségi és beszállítói biztonsági követelményeket.

DORA. A Digital Operational Resilience Act a pénzügyi szektor digitális működési ellenálló képességét szabályozza. 2025. január 17-től alkalmazandó. Fő pillérei: IKT-kockázatkezelés, incidensjelentés, digitális működési reziliencia tesztelése, harmadik fél IKT-kockázat kezelése és információmegosztás.

Magyar kiberbiztonsági szabályozás. Magyarországon a NIS2-höz kapcsolódó jelenlegi központi jogszabály a Magyarország kiberbiztonságáról szóló 2024. évi

LXIX. törvény és annak végrehajtási rendeletei, például a 418/2024. (XII. 23.) Korm. rendelet. Ezek az elektronikus információs rendszerek kockázatarányos védelméhez, hatósági felügyeletéhez, nyilvántartási és auditkövetelményeihez kapcsolódnak.

Szabványok és ajánlások

- **ISO/IEC 27001:2022:** információbiztonsági irányítási rendszer követelményei; kockázatalapú, tanúsítható menedzsmentrendszer.
- **ISO/IEC 27002:2022:** információbiztonsági kontrollok gyakorlati katalógusa.
- **ISO/IEC 27005:** információbiztonsági kockázatmenedzsment.
- **ISO/IEC 27035:** információbiztonsági incidenskezelés.
- **ISO/IEC 27701:** adatvédelmi információmenedzsment, GDPR-megfelelés támogatása.
- **NIST Cybersecurity Framework 2.0:** Govern, Identify, Protect, Detect, Respond, Recover funkciók köré szervezett kiberbiztonsági keretrendszer.
- **NIST SP 800-61 Rev. 3:** incidensválasz ajánlások a kiberbiztonsági kockázatmenedzsment részeként; 2025-ben véglegesített revízió, amely felváltotta a Rev. 2-t.
- **NIST SP 800-63:** digitális identitás és felhasználó-hitelesítés irányelvei.
- **PCI DSS:** fizetési kártyaadatok kezelésére vonatkozó iparági követelményrendszer.

A szabványok és rendeletek viszonya: a jogszabály kötelező követelményt ad, a szabvány pedig módszertant, kontrollkatalógust és bizonyítási keretet. Egy szervezet például ISO/IEC 27001 szerinti ISMS-sel tudja megszerezni a GDPR, NIS2 vagy DORA által megkövetelt kockázatkezelési, naplózási és incidenskezelési kontrollokat.

1.3. 3. tétel: gépi tanulási alapismeretek

Felügyelt és nem felügyelt tanulás; egy- és többváltozós lineáris regresszió; gradiens csökkenési eljárás; sztochasztikus és mini-batch gradiens csökkentés; leírók normalizálása; polinomiális regresszió; normál egyenlet; logisztikus regresszió; két- és többosztályos osztályozás; regularizáció (alul- és túltanulás); regularizált lineáris és logisztikus regresszió; neurális hálók; backpropagation algoritmus; numerikus gradiensellenőrzés.

Felügyelt és nem felügyelt tanulás

Felügyelt tanulásnál a tanítóadatok címkézettek: adottak $(x^{(i)}, y^{(i)})$ párok. A cél olyan h_θ modell tanulása, amely új bemenetre jól becsli a kimenetet. Két alapfeladat:

- **Regresszió:** folytonos érték becslése, például lakásár vagy hőmérséklet.
- **Osztályozás:** diszkrét kategória becslése, például spam/nem spam vagy beteg/nem beteg.

Nem felügyelt tanulásnál nincs előre megadott címke. A cél az adatok belső

szerkezetének feltárása, például klaszterezéssel, dimenziócsökkentéssel vagy anomáliadetektálással.

Lineáris regresszió

Az egyváltozós lineáris regresszió egyetlen bemeneti változóból becsül folytonos ki-
menetet:

$$\hat{y} = h_{\theta}(x) = \theta_0 + \theta_1 x.$$

Többsváltozós esetben több jellemzőt használunk:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T x,$$

ahol $x_0 = 1$. A négyzetes hiba költségfüggvénye:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2.$$

A modell tanításakor a költségfüggvény minimumát keressük. A gradiens megad-
ja, merre nő leggyorsabban a hiba, ezért a paramétereket ennek ellenkező irányába
frissítjük. Batch gradiens csökkenésnél minden iterációban a teljes tanítóhalmazon
számoljuk a gradienst:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Itt α a tanulási ráta; túl nagy értéknél a módszer divergálhat, túl kicsinél lassan
konvergál.

Sztochasztikus és mini-batch gradiens csökkenés

Sztochasztikus gradiens csökkenésnél minden lépésben egyetlen tanítópélda
alapján frissítünk:

$$\theta_j := \theta_j - \alpha \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Előnye, hogy nagy adathalmaznál gyorsan ad haladási irányt; hátránya, hogy zajo-
sabb, ezért a költségfüggvény nem feltétlenül csökken minden egyes lépésben.

Mini-batch gradiens csökkenésnél b darab példát használunk egy frissítéshez:

$$\theta_j := \theta_j - \alpha \frac{1}{b} \sum_{k=i}^{i+b-1} \left(h_{\theta}(x^{(k)}) - y^{(k)} \right) x_j^{(k)}.$$

Ez kompromisszum a batch és az SGD között, és modern hardveren jól párhuzamo-
sítható.

Leírók normalizálása és polinomiális regresszió

A gradiens módszerek gyorsabban konvergálnak, ha a jellemzők hasonló skálán vannak. Gyakori normalizálás:

$$x_j^{(i)} := \frac{x_j^{(i)} - \mu_j}{s_j},$$

ahol μ_j a j -edik jellemző átlaga, s_j pedig szórása vagy terjedelme. Erre különösen akkor van szükség, ha például egyik jellemző néhány egység, a másik több ezres nagyságrendű.

Polinomiális regressziónál új jellemzőket képzünk a meglévőkből, hogy nemlineáris kapcsolatot is lineáris regressziós modellbe írassunk:

$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3.$$

Ilyenkor a skálázás különösen fontos, mert x , x^2 és x^3 nagyságrendje nagyon eltérhet.

Normál egyenlet

Lineáris regresszió

nál az optimális paraméter iteráció nélkül is számítható:

$$\theta = (X^T X)^{-1} X^T y.$$

Előnye, hogy nem kell tanulási rátát választani és nincs iteratív konvergencia. Hátránya, hogy sok jellemző esetén a mátrixinverzió drága; ha $X^T X$ nem invertálható, pszeudoinverz használható.

Logisztikus regresszió

Bináris osztályozásnál $y \in \{0, 1\}$, ezért nem közvetlenül lineáris regressziót használunk, hanem valószínűséget becsülünk:

$$h_\theta(x) = g(\theta^T x), \quad g(z) = \frac{1}{1 + e^{-z}}.$$

A sigmoid függvény bármely valós számot a $(0, 1)$ intervallumra képez le. Alapértelmezett döntési szabály:

$$h_\theta(x) \geq 0.5 \Rightarrow \hat{y} = 1, \quad h_\theta(x) < 0.5 \Rightarrow \hat{y} = 0.$$

Az összevont cross-entropy költségfüggvény:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right].$$

Ha $y = 1$, akkor a második tag kiesik; ha $y = 0$, akkor az első tag esik ki. A gradiens alakja formailag hasonló a lineáris regresszióéhoz:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Két- és többosztályos osztályozás

Kétosztályos osztályozásnál egyetlen bináris döntést hozunk. Többosztályos esetben a **one-vs-all** módszer K osztályra K darab bináris osztályozót tanít:

$$h_{\theta}^{(k)}(x) = P(y = k \mid x; \theta), \quad k = 1, \dots, K.$$

Predikciókor azt az osztályt választjuk, amelyre a becsült valószínűség legnagyobb:

$$\hat{y} = \arg \max_k h_{\theta}^{(k)}(x).$$

Neurális hálónál többosztályos osztályozásra gyakori a softmax kimenet.

Regularizáció

Alultanulás esetén a modell túl egyszerű, ezért a tanító- és validációs hiba is magas. Ez magas bias jelenség. **Túltanulás** esetén a modell túl jól illeszkedik a tanítóadat zajára, ezért a tanító hiba alacsony, de a validációs hiba magas. Ez magas variance jelenség.

A túltanulás kezelésének két tipikus módja a jellemzők számának csökkentése és a regularizáció. Regularizációval büntetjük a nagy paramétereket, így egyszerűbb, jobban általánosító modellt kapunk. A λ regularizációs paraméter szabályozza a büntetés erősségét: túl nagy értéknél alultanulás, túl kicsinél túltanulás maradhat.

Regularizált lineáris regressziónál:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2.$$

Regularizált logisztikus regressziónál:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2.$$

Fontos, hogy a bias tagot, vagyis θ_0 -t általában nem regularizáljuk.

Neurális hálók és backpropagation

A neurális hálók rétegelt struktúrájú modellek, amelyek nemlineáris összefüggések tanulására alkalmasak. Fő részeik:

- **Bemeneti réteg:** a jellemzőket fogadja.
- **Rejtett rétegek:** nemlineáris transzformációkat tanulnak; több rejtett réteg esetén mély neurális hálóról beszélünk.
- **Kimeneti réteg:** regressziós értéket, bináris kimenetet vagy osztályvalószínűségeket ad.

Egy neuron a bemenetek súlyozott összegét számítja ki, majd aktivációs függvényt alkalmaz:

$$z = w^T x + b, \quad a = \phi(z),$$

ahol ϕ lehet például sigmoid, tanh vagy ReLU. **Forward propagation** során a be-menetet rétegről rétegre előre felé számítjuk, amíg megkapjuk a predikciót.

Backpropagation során a veszteség gradienst a kimeneti rétegtől visszafelé számítjuk a láncszabály segítségével. Egy egyszerű lokális derivált:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}.$$

A paraméterfrissítés például gradiens csökkenéssel:

$$w := w - \alpha \frac{\partial L}{\partial w}, \quad b := b - \alpha \frac{\partial L}{\partial b}.$$

Numerikus gradiensenőrzés

A numerikus gradiensenőrzés célja annak ellenőrzése, hogy a backpropagation implementáció helyesen számolja-e a gradienst. Egy paraméterre központi differenciával:

$$\frac{\partial J(\theta)}{\partial \theta_j} \approx \frac{J(\theta_1, \dots, \theta_j + \varepsilon, \dots) - J(\theta_1, \dots, \theta_j - \varepsilon, \dots)}{2\varepsilon}.$$

Ezt összehasonlítjuk a backpropagation által adott gradienssel. Ha a relatív eltérés kicsi, például 10^{-7} nagyságrendű, akkor az implementáció valószínűleg helyes:

$$\frac{\|g_{\text{num}} - g_{\text{backprop}}\|}{\|g_{\text{num}}\| + \|g_{\text{backprop}}\|} \ll 1.$$

Fontos, hogy a numerikus gradiensenőrzés lassú, ezért csak hibakeresésre használjuk, tanítás közben nem.

2. Az adattudomány gyakorlati hátteréhez kapcsolódó alapismeretek.

2.1. 4. tétel: modellkiértékelés, SVM, klaszterezés, dimenziócsökkentés, ajánlórendszerek és MapReduce

Tanító/teszt/validációs adatfelbontás; tanítási diagnosztika; tanulási görbék; hibamérés és kiegyenlített osztályok; támasztóvektor-gépek és magfüggvények; klaszterezés; klaszterek számának meghatározása; dimenziócsökkentés; anomáliadetektálás normális eloszlással; ajánlórendszerek; tartalom alapú ajánlás; kollaboratív szűrés; MapReduce és párhuzamosítás.

Tétel fókusza

A tétel a gépi tanulási modellek gyakorlati kiértékelését, diagnosztikáját és néhány klasszikus módszerét foglalja össze. Vizsgán érdemes minden témát ugyanarra a logikára felfűzni: *mit tanulunk, hogyan mérjük, mikor hibázik, és milyen gyakorlati döntést*

támogat. A legfontosabb gondolat, hogy a jó tanítóhiba önmagában nem bizonyít jó modellt; a validáció, a tesztelés, a megfelelő metrika és az adatszivárgás elkerülése legalább ilyen fontos.

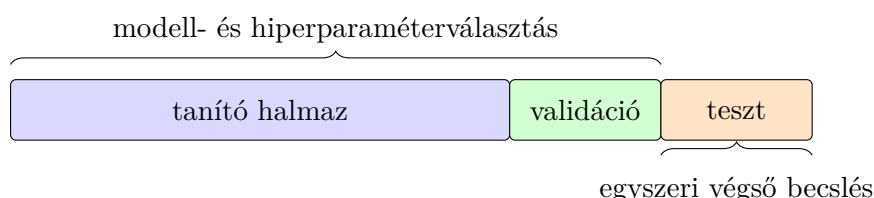
Elhatárolás az 5. és 6. tételtől. Az optimalizálási algoritmusokat itt csak annyiban említjük, amennyiben a modellek tanításához vagy skálázásához szükségesek; a gradiens-, Newton-, kvázi-Newton-, konjugált gradiens és sztochasztikus optimalizálási részletek az 5. tételhez tartoznak. A PCA-t, normális eloszlást és többdimenziós statisztikai módszereket a 4. tételben alkalmazási szinten használjuk; a főkomponens-analízis matematikai háttere, faktoranalízis, kanonikus korrelációanalízis, diszkriminancia-analízis és többdimenziós skálázás a 6. tétel témája.

Tanító, validációs és teszt adatfelbontás

A tanító/validációs/teszt felosztás célja, hogy a modell teljesítményét ne azon az adaton becsüljük, amelyen a paramétereit vagy a hiperparamétereit kiválasztottuk.

- **Tanító halmaz:** ezen tanulnak a modell paramétereit, például regressziós súlyok, neurális háló súlyai, SVM szeparáló hipersíkja.
- **Validációs halmaz:** modellválasztásra és hiperparaméter-hangolásra szolgál, például polinomfok, regulárizációs paraméter, SVM kernelparaméter, döntési küszöb.
- **Teszt halmaz:** csak a végső modell egyszeri, független értékelésére használható.

Tipikus arányok: 60/20/20, 70/15/15 vagy nagy adathalmaznál akár 98/1/1. Kiegyenlített osztályoknál **rétegzett** felosztás szükséges, hogy minden részhalmazban hasonló legyen az osztályarány. Idősoros adatoknál nem szabad véletlenül keverni: a múltból tanítunk, későbbi időszakon validálunk és tesztelünk.



1. ábra. A teszthalmazt nem használjuk modellválasztásra, különben optimista teljesítménybecslést kapunk.

Kis adathalmaznál használható **k -szoros keresztvalidáció:** az adatot k részre osztjuk, mindig $k - 1$ részen tanítunk és egy részen validálunk. A teszthalmaz ettől még külön marad, ha végső általánosítási hibát akarunk becsülni.

Gyakori munkamenet: először a tanító és validációs részen választunk modellt és hiperparamétert, majd a kiválasztott beállítással sokszor újratanítunk a tanító+validációs adaton, és végül egyszer mérünk a teszten. A teszt eredményét már nem használjuk újabb döntésekhez.

Tanítási diagnosztika

A tanítási diagnosztika azt vizsgálja, hogy a rossz teljesítmény oka alultanulás, túltanulás, adatprobléma, rossz metrika vagy rosszul választott hiperparaméter-e.

Bias–variance diagnózis Legyen J_{train} a tanító hiba és J_{val} a validációs hiba.

Jelenség	Hibatípus	Teendő
J_{train} magas, J_{val} magas	magas bias, alultanulás	bonyolultabb modell, jobb jellemzők, kisebb regulárizáció
J_{train} alacsony, J_{val} magas	magas variance, túltanulás	több adat, erősebb regulárizáció, egyszerűbb modell
Mindkettő alacsony	jó illeszkedés	végző tesztelés

A regulárizációs paraméter hatása tipikusan:

- túl nagy λ : túl egyszerű modell, magas bias;
- túl kicsi λ : túl rugalmas modell, magas variance;
- köztes λ : jobb általánosítás.

Gyakori hibaforrások

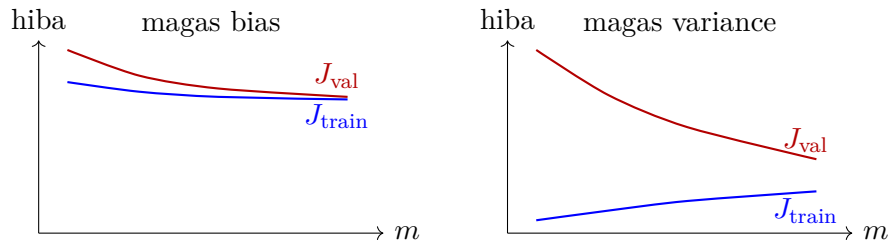
- **Adatszivárgás:** a tanítás során olyan információ jut a modellhez, amely predikciókor nem lenne elérhető.
- **Nem megfelelő split:** idősoros vagy csoportos adatok véletlen keverése.
- **Skálázás rossz helyen:** a standardizálás paramétereit csak a tanító adaton szabad illeszteni, majd ugyanazt alkalmazni validációra/tesztre.
- **Rossz metrika:** kiegyenlítetlen osztályoknál az accuracy félrevezető lehet.
- **Eloszláseltolódás:** a teszt vagy éles adat más eloszlásból jön, mint a tanító adat.

Tanulási görbék

A tanulási görbe a tanítóhalmaz méretének függvényében ábrázolja a tanító és validációs hibát. A tanító hiba általában nőhet a tanítóhalmaz növelésével, mert a modellnek többféle példára kell illeszkednie. A validációs hiba általában csökken, amíg a több adat segíti az általánosítást.

- **Magas bias:** nagy adatmennyiségnél is magas J_{train} és J_{val} , a két görbe közel kerül egymáshoz. Több adat önmagában kevésbé segít.
- **Magas variance:** J_{train} alacsony, J_{val} magas, nagy rés van köztük. Több adat, regulárizáció vagy egyszerűbb modell segíthet.
- **Jó modell:** a validációs hiba alacsony szinten stabilizálódik, a train–validation rés nem nagy.

Tanulási görbékkel eldönthető, hogy érdemes-e további adatot gyűjteni, vagy inkább modellkapacitást, jellemzőket, regulárizációt és adatminőséget kell javítani.



2. ábra. Tanulási görbék sematikusan: bias esetén a két hiba magas szinten összezár, variance esetén nagy a rés a tanító és validációs hiba között.

Hibamérés és kiegyenlítetlen osztályok

Kiegyenlítetlen osztályoknál az egyik osztály ritka. Például ha a pozitív osztály aránya 1%, akkor egy mindig negatív predikciót adó modell 99% accuracyt ér el, miközben semmit nem ismer fel.

Konfúziós mátrix

	Valós pozitív	Valós negatív
Prediktált pozitív	TP	FP
Prediktált negatív	FN	TN

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}},$$

$$\text{Specificity} = \frac{TN}{TN + FP},$$

$$\text{Balanced accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}.$$

Magas **precision**: amit pozitívnak jelölünk, az nagy eséllyel tényleg pozitív. Magas **recall**: kevés valódi pozitívat hagyunk ki. Ritka pozitív osztálynál gyakran a PR-görbe és PR-AUC informatívabb, mint a ROC-AUC.

Vizsgán hasznos különbség: a ROC-görbe a TPR–FPR kompromisszumot mutatja, míg a PR-görbe közvetlenül a precision–recall kompromisszumot. Erősen ritka pozitív osztálynál a PR-görbe érzékenyebben jelzi, hogy a pozitív predikciók tényleg értékesek-e. A döntési küszöb nem szükségszerűen 0.5: üzleti költség alapján lehet magasabb precisiót vagy magasabb recallt választani.

Kezelési lehetőségek:

- osztálysúlyozás vagy költségérzékeny tanítás;
- túlmintavételezés, alulmintavételezés, SMOTE;
- döntési küszöb validációs halmazon való beállítása;
- megfelelő metrika választása: F1, recall@precision, balanced accuracy, MCC.

Támasztóvektor-gépek és magfüggvények

Lineáris SVM Az SVM bináris osztályozó, ahol $y_i \in \{-1, +1\}$. Lineáris esetben a döntési függvény:

$$f(x) = w^T x + b, \quad d(x) = \text{sign}(f(x)).$$

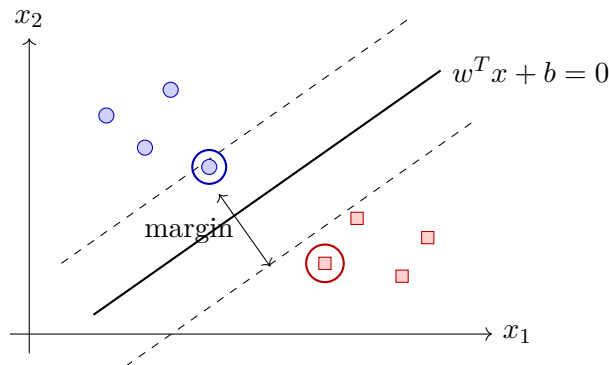
A cél olyan szeparáló hipersík választása, amely maximalizálja az elválasztó sávot, vagyis a margin-t. Lineárisan szeparálható esetben:

$$y_i(w^T x_i + b) \geq 1.$$

A margin szélessége $\frac{2}{\|w\|}$, ezért ekvivalens feladat:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{feltéve, hogy} \quad y_i(w^T x_i + b) \geq 1.$$

A **támasztóvektorok** azok a tanítópontok, amelyek a margint meghatározzák; ezekre a feltétel egyenlőséggel teljesül.



3. ábra. Lineáris SVM: a döntési hipersíkot a marginon fekvő támasztóvektorok határozzák meg.

Soft margin SVM Valós adatoknál gyakran nincs tökéletes lineáris szeparáció. Ekkor slack változókat vezetünk be:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

A C paraméter a margin szélessége és a hibák büntetése közötti kompromisszum:

- nagy C : kevés tanítási hiba, de nagyobb túltanulási kockázat;

- kis C : szélesebb margin, több megengedett hiba, erősebb regulárizáció.

Másik nézőpontból az SVM a **hinge loss** regularizált minimalizálása:

$$\sum_{i=1}^m \max(0, 1 - y_i(w^T x_i + b)) + \lambda \|w\|^2.$$

Ha egy pont jó oldalon és a marginon kívül van, a vesztesége nulla; ha rossz oldalon vagy a marginon belül van, büntetést kap.

Kernel trick Nemlineáris osztályozásnál az adatot implicit módon magasabb dimenziós térbe képezzük:

$$K(x, z) = \phi(x)^T \phi(z).$$

A kernel trick lényege, hogy a skalárszorzatot közvetlenül kernelértékkel számoljuk, nem kell explicit előállítani $\phi(x)$ -et.

Gyakori kernelek:

- **Lineáris:** $K(x, z) = x^T z$.
- **Polinomiális:** $K(x, z) = (x^T z + c)^d$.
- **Gauss/RBF:**

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) = \exp(-\gamma\|x - z\|^2).$$

Az RBF-kernelben a γ nagy értéke lokális, hullámosabb döntési határt eredményezhet, ami túltanuláshoz vezethet; kis γ simább határt adhat, de alultanulhat. Az SVM érzékeny a skálázásra, ezért a jellemzőket általában standardizálni kell. Sok osztály esetén one-vs-rest vagy one-vs-one kiterjesztés használható.

Klaszterezés

A klaszterezés felügyeletlen tanulás: a megfigyelések nincsenek címkézve, a cél hasonló elemek csoportosítása. Alapja valamilyen távolság vagy hasonlóság.

Gyakori távolságok:

- Euklideszi: $\|x - y\|_2$.
- Manhattan: $\|x - y\|_1$.
- Minkowski: $\|x - y\|_q$.
- Mahalanobis: $(x - y)^T S^{-1}(x - y)$.
- Koszinusz-távolság szöveges vagy ritka vektoroknál.

K-közép módszer A k-means célja K darab klaszter és centroid meghatározása úgy, hogy a klaszteren belüli négyzetes eltérés minimális legyen:

$$W(C_1, \dots, C_K) = \sum_{j=1}^K \sum_{x \in C_j} \|x - \mu_j\|^2.$$

Algoritmus:

1. Válasszunk K kezdő centroidot.
2. Minden pontot rendeljünk a legközelebbi centroidhoz.
3. Frissítsük a centroidokat a klaszterek átlagaként.
4. Ismételjük, amíg nincs érdemi változás.

Előnye: gyors, egyszerű, nagy adaton is jól skálázódik. Hátránya: meg kell adni K -t, érzékeny a kezdőpontokra, a skálázásra és a kiugró értékekre, főleg gömbszerű klasztereket talál.

Hierarchikus klaszterezés Agglomeratív esetben kezdetben minden pont külön klaszter, majd mindig a két legközelebbi klasztert vonjuk össze. A folyamat dendrogrammal ábrázolható. Klasztertávolságok:

- single linkage: két klaszter legközelebbi pontjainak távolsága;
- complete linkage: két klaszter legtávolabbi pontjainak távolsága;
- average linkage: pontpárok átlagos távolsága;
- centroid módszer: klaszterközéppontok távolsága;
- Ward-módszer: klaszteren belüli variancia növekedését minimalizálja.

Klaszterek számának meghatározása

Nincs univerzálisan helyes klaszterszám, mert a klaszter fogalma függ a távolságtól, skálázástól és üzleti céltól.

- **Könyök módszer:** ábrázoljuk a klaszteren belüli hibát K függvényében. Ahol a csökkenés megtörik, ott lehet jó K .
- **Silhouette mutató:**

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))},$$

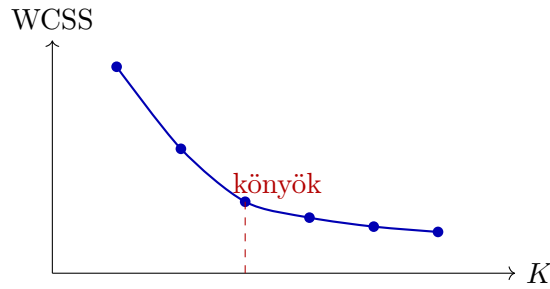
ahol $a(i)$ a pont saját klaszterén belüli átlagos távolsága, $b(i)$ a legközelebbi másik klaszterhez mért átlagos távolsága. Értéke $[-1, 1]$, nagyobb jobb.

- **Gap statistic:** a megfigyelt klaszterezési hibát véletlen referenciaeloszláshoz hasonlítja.
- **Dendrogram vágása:** hierarchikus klaszterezésnél ott vágunk, ahol nagy távolságugrás látszik.
- **Doméntudás:** például ügyfélszegmenseknél az üzletileg kezelhető klaszterszám is korlát.

Fontos, hogy ezek a módszerek nem automatikus igazságok: több jelölt K -t érdemes összevetni stabilitással, interpretálhatósággal és a felhasználási céllal.

Dimenziócsökkentés

A dimenziócsökkentés a 4. tételben **alkalmazott gépi tanulási eszköz:** kevesebb jellemzővel szeretnénk dolgozni úgy, hogy a modellezéshez fontos információ nagy része megmaradjon. Tipikus célok:



4. ábra. Könyök módszer: azt a K értéket keressük, ahol a klaszteren belüli hiba csökkenése látványosan lassul.

- zaj és redundancia csökkentése;
- gyorsabb tanítás és kisebb tárigény;
- vizualizáció két vagy három dimenzióban;
- klaszterezés vagy ajánlórendszer előtti tömörítés.

PCA alkalmazói szinten A PCA lineáris módszer: új, egymásra merőleges koordinátatengelyeket keres, ahol az első irány hordozza a legnagyobb variációt, a második a maradékból a legtöbbet, és így tovább. Gyakorlati pipeline:

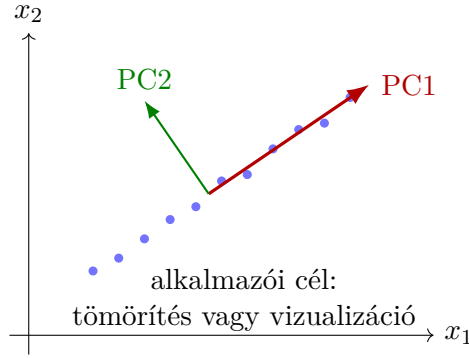
1. csak a tanító adaton illesztjük a skálázást és a PCA-transzformációt;
2. kiválasztjuk a megtartott komponensek számát, például explained variance alapján;
3. ugyanazt a transzformációt alkalmazzuk validációs, teszt és éles adatra;
4. ellenőrizzük, hogy a dimenziócsökkentés javítja-e a validációs teljesítményt vagy a futási időt.

$$z = U_k^T(x - \mu), \quad \text{megtartott variancia aránya} = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^p \lambda_j}.$$

Itt U_k az első k főirányt tartalmazza, λ_j pedig az adott főkomponenshez tartozó variancia. A PCA matematikai háttere, a sajátérték-felbontás részletes tárgyalása és a többváltozós statisztikai kapcsolatok a 6. tételhez tartoznak.

Módszerválasztás

- **PCA:** gyors, lineáris, jól használható zajcsökkentésre és tömörítésre.
- **t-SNE, UMAP:** főleg feltáró vizualizációra alkalmas; a tengelyek nem feltétlenül értelmezhetők közvetlenül.
- **Autoencoder:** neurális hálós, nemlineáris tömörítés; nagyobb adatigényű, de komplex mintázatokat is tanulhat.
- **Jellemzőválasztás:** nem új tengelyeket képez, hanem eredeti változókat tart meg; interpretálhatóbb lehet.



5. ábra. PCA alkalmazói szemléletben: nagy varianciájú irányokra vetítjük az adatot.

A 4. tételben a lényeg nem a PCA levezetése, hanem az, hogy a dimenziócsökkentést mindig validációval kell ellenőrizni: javítja-e a modell általánosítását, futási idejét vagy áttekinthetőségét. Ha rontja a validációs metrikát, akkor hiába őriz meg sok varianciát, a konkrét feladatra nem hasznos.

Anomáliadetektálás normális eloszlással

Anomáliadetektálásnál tipikusan sok normális példánk van, kevés vagy nincs címkézett anomália. A 4. tételben a normális eloszlást gyakorlati detektálási modellként használjuk: az alacsony valószínűségű megfigyeléseket jelöljük gyanúsak. A többdimenziós normális eloszlás részletes statisztikai háttere a 6. tételhez tartozik.

Egy jellemzőre a Gauss-eloszlás:

$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Paraméterbecslés:

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}, \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2.$$

Több jellemzőnél egyszerű, alkalmazói modellként feltételezhetjük, hogy a jellemzők feltételesen függetlenek:

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2).$$

Döntési szabály:

$$p(x) < \varepsilon \Rightarrow \text{anomália}, \quad p(x) \geq \varepsilon \Rightarrow \text{normális}.$$

Az ε küszöböt validációs halmazon választjuk, például F1 alapján. Ha nincs címkézett anomália, akkor a küszöböt üzleti kockázat vagy elvárt riasztási arány szerint lehet beállítani, de ezt később valódi visszajelzéssel ellenőrizni kell.

Gyakorlati megjegyzések:

- A paramétereket főleg normális példákából becsüljük; a validációs halmazon jó, ha van néhány ismert anomália a küszöbhangoláshoz.
- Ha egy jellemző erősen ferde eloszlású, logaritmikus vagy Box–Cox jellegű transzformáció segíthet Gauss-szerűbbé tenni.
- Erősen korrelált jellemzőknél a függetlenségi feltételezés gyenge lehet; ilyenkor a részletesebb többváltozós normális modell már a 6. tétel témája.

Tipikus alkalmazások: csalásdetektálás, hibás szenzorértékek, gépállapot-monitorozás, hálózati behatolás előszűrése.

Ajánlórendszerek

Az ajánlórendszerek célja releváns elemek ajánlása felhasználóknak: film, termék, zene, hír, kurzus. Bemenet lehet explicit értékelés, például 1–5 csillag, vagy implicit jelzés, például kattintás, vásárlás, megtekintési idő.

Fő problémák:

- **Cold start:** új felhasználóról vagy új elemről kevés adat van.
- **Sparsity:** a felhasználó–elem mátrix ritka.
- **Popularity bias:** a népszerű elemek túl gyakran jelennek meg.
- **Értékelési torzítás:** nem minden hiányzó érték jelent negatív preferenciát.

Tartalom alapú ajánlás Tartalom alapú ajánlásnál az elem jellemzőiből indulunk ki. Például filmnél műfaj, rendező, szereplő, kulcsszavak; szövegnél TF-IDF vagy embedding.

Egyszerű modellben egy felhasználóhoz tanulhatunk θ_u preferenciavektort, az elemhez pedig adott az x_i jellemzővektor:

$$\hat{r}_{ui} = \theta_u^T x_i.$$

Ez ugyanarra emlékeztet, mint egy regressziós modell: azt becsüljük, mennyire fogja az u felhasználó kedvelni az i elemet.

Lépések:

1. Elemjellemzők előállítása.
2. Felhasználói profil tanulása a korábbi kedvelt elemekből.
3. Hasonlóság számítása a profil és új elemek között.
4. Legmagasabb pontszámú elemek ajánlása.

Előnye, hogy új elem is ajánlható, ha ismerjük a tartalmát. Hátránya, hogy bezárhatja a felhasználót a korábbi érdeklődési körébe, és nehezebben fedez fel meglepő, de releváns elemeket.

Kollaboratív szűrés Kollaboratív szűrésnél nem az elemek tartalmát, hanem a felhasználók viselkedési mintázatait használjuk.

Memóriaalapú módszerek:

- user-user: hasonló felhasználók kedvelt elemeit ajánlja;
- item-item: a felhasználó által kedvelt elemekhez hasonló elemeket ajánl.

Item-item módszerrel például az ajánlás lényege: ha két elem sok felhasználónál hasonló értékelési mintázatot mutat, akkor aki az egyiket kedvelte, annak a másik is ajánlható. Hasonlóság lehet koszinusz-hasonlóság vagy Pearson-korreláció.

Mátrixfaktorizáció: Legyen R a felhasználó–elem értékelési mátrix. A cél:

$$R_{ui} \approx p_u^T q_i,$$

ahol p_u a felhasználó, q_i az elem látens vektora. Gyakori tanítási célfüggvény:

$$\min_{p,q} \sum_{(u,i) \in \Omega} (R_{ui} - p_u^T q_i)^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right).$$

Előnye, hogy tartalmi jellemzők nélkül is működik és képes rejtett preferenciákat tanulni. Hátránya az új felhasználó/új elem probléma és a ritka mátrix. Gyakori megoldás a hibrid ajánlás, amely tartalmi és kollaboratív jeleket kombinál.

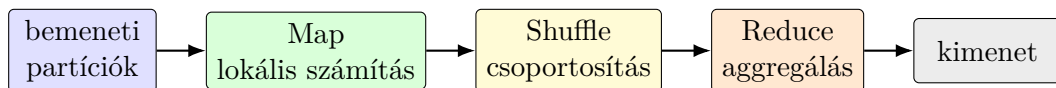
MapReduce és párhuzamosítás

A MapReduce nagy adatok elosztott feldolgozására szolgáló programozási modell. Két fő lépése:

- **Map:** bemeneti rekordokból kulcs–érték párokat állít elő.
- **Reduce:** az azonos kulcshoz tartozó értékeket összesíti.

Általános séma:

$$\text{map}(k_1, v_1) \rightarrow [(k_2, v_2)], \quad \text{reduce}(k_2, [v_2]) \rightarrow [(k_3, v_3)].$$



6. ábra. MapReduce folyamat: az adatot lokálisan feldolgozzuk, kulcs szerint átrendezzük, majd aggregáljuk.

Példa szószámlálásra:

- Map: minden szóra kiadja (szó, 1).
- Shuffle: azonos szavakat egy reducerhez csoportosít.
- Reduce: összeadja az értékeket.

Gépi tanulásban párhuzamosítható:

- veszteség-, metrika- vagy modellfrissítési részösszegek számítása adatrészeken;
- k-means hozzárendelési lépése és klaszterösszegek aggregálása;

- ajánlórendszerben nagy felhasználó–elem mátrix feldolgozása;
- logfeldolgozás és feature engineering.

Korlátok:

- iteratív algoritmusoknál sok MapReduce kör kell, ami drága;
- shuffle hálózati költsége magas lehet;
- adatferdeség esetén néhány reducer túlterhelődik;
- nem minden algoritmus párhuzamosítható hatékonyan.

Modern környezetben a MapReduce elvét gyakran Spark-szerű memóriabeli ke-
retrendszerek váltják ki, de az alapgondolat ugyanaz: az adatot partíciókra bontjuk,
lokálisan számolunk, majd aggregálunk.